

ORBIT PHILOSOPHY

첫 결과물이 곧 완성은 아니다

첫 프롬프트와 MVP 이후, 인간과 LLM이 함께 완성에 가는 법

첫 결과물이 곧 완성은 아니다

첫 프롬프트와 MVP 이후,
인간과 LLM이 함께 완성해 가는 법

ORBIT Philosophy

PDF PROTOTYPE · v0.3 · 2026-09-30

전체 구성

1부. 완성처럼 보이는 첫 결과물

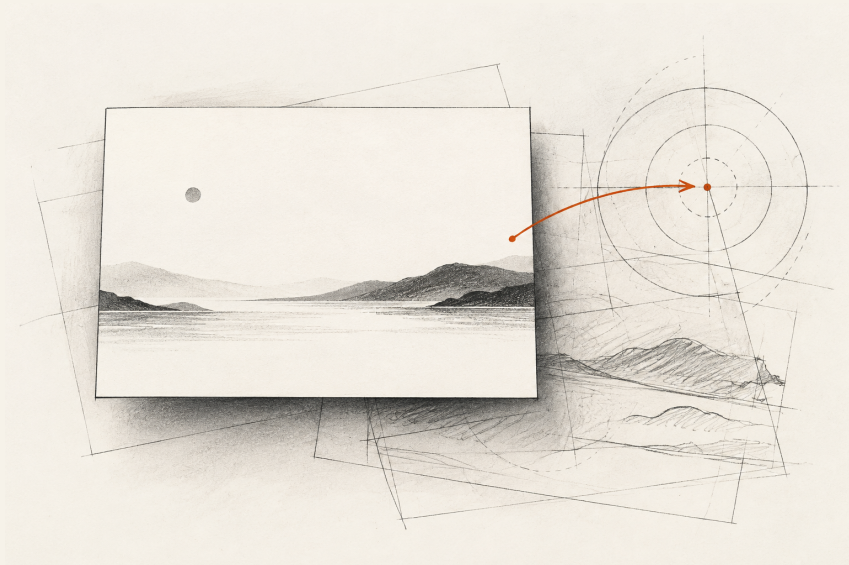
1. 완성처럼 보이는 첫 결과물
2. 친절한 답이 목표를 잃게 할 때
3. 같은 말을 해도 같은 뜻이 아니다

2부. 대화를 판단의 과정으로 바꾸기

4. 대화 깔때기와 통역
5. 다름을 지우지 않는 관계
6. 판단권과 파트너의 책임
7. 선택에는 이유가 따라야 한다

3부. 다음 판단을 처음부터 시작하지 않기

8. 대화가 사라지면 관계도 처음으로 돌아간다
9. 오해와 교정의 궤적을 남기는 법
10. 완성하지 않고도 완결할 수 있는가



완성되어 보이는 결과도 목표와 맞는지 다시 확인해야 한다.

이 판본에 대하여

이 파일은 《첫 결과물이 곧 완성은 아니다》의 출간본이 아니다. 제목과 구성, 읽기 경험을 함께 검토하기 위한 PDF 시제품이다. 현재 3부 10장의 설계와 1부의 세장을 담았다.

이 책은 한 번에 좋은 프롬프트를 만드는 기술보다 첫 결과물 이후의 판단을 다룬다. 첫 응답이나 첫 MVP가 완성처럼 보여도, 인간과 LLM은 목표·경계·완료 조건을 서로 다르게 이해할 수 있다. 그 차이를 어떻게 확인하고 교정하며 다음 판단을 위해 무엇을 기록할지 묻는다.

제1장 — 완성처럼 보이는 첫 결과물

프롬프트를 입력하고 몇 초만 기다리면 결과가 나타난다. 문장은 제목과 소제목을 갖추고, 코드는 실행되며, 계획표에는 해야 할 일이 순서대로 놓인다. 첫 프롬프트에서 곧바로 나온 답일 수도 있고, 몇 차례 대화를 거쳐 만든 첫 MVP일 수도 있다. 아직 충분히 검토하지 않았는데도 결과물은 이미 완성된 얼굴을 하고 있다.

이 책에서 말하는 첫 결과물은 바로 이 지점을 가리킨다. 첫 응답 하나만이 아니라, LLM과의 초기 대화를 통해 처음으로 형태를 갖춘 문서·코드·디자인·계획·MVP를 모두 포함한다.

첫 결과물이 주는 가장 큰 착각은 단지 “빠르다”는 느낌이 아니다. 형태가 갖춰졌으니 판단도 끝났다고 여기는 것이다.

우리의 대화에서는 이러한 태도를 인스턴트적 사고라고 불렀다.

“내가 말한 인스턴트적 사고란, 프롬프트를 입력하면 완성된 결과물이 곧바로 나온다고 여기는 생각이다.”

편집 주: 대화 원문의 의미를 유지하며 문장을 다듬었다.

이 말은 빠른 생성 자체를 거부한다는 뜻이 아니다. 느리게 만들어야 더 가치 있다는 주장도 아니다. 문제는 결과물이 나온 속도가 아니라, 결과물이 나온 순간 판단까지 멈추는 데 있다.

완성된 외형이 판단을 앞지를 때

사람이 직접 초안을 쓰면 미완성의 흔적이 눈에 보인다. 빈 문단이 있고, 어색한 문장이 있으며, 아직 결정하지 못한 부분에는 표시가 남는다. 그러나 LLM은 빈자리를 자연스러운 문장으로 채운다. 무엇을 모르거나 잘못 이해했는지조차 매끄러운 표현 속에 가릴 수 있다.

그래서 첫 결과물은 작업을 빠르게 시작하게 하면서도, 아직 확인해야 할 것이 남았다는 사실을 감춘다.

문서가 매끄럽다는 것은 목표를 정확히 이해했다는 증거가 아니다. 기능이 작동한다는 것은 사용자가 제기한 문제가 해결됐다는 증거가 아니다. 정보가 많다는 것은 중요한 것이 지켜졌다는 증거가 아니다. 그러나 완성된 외형을 만나면 우리는 이 차이를 쉽게 잊는다.

특히 LLM을 처음 사용하는 사람에게는 이 인상이 더 강하다. 오랫동안 고민해야 만들 수 있었던 형태가 순식간에 나타나기 때문이다. 놀라움은 자연스럽다. 다만 그 놀라움이 다음 질문을 대신해서는 안 된다.

MVP는 완료 선언이 아니라 검증의 시작이다

MVP도 같은 오해를 받는다. 최소한의 기능이 작동하면 이제 세부만 다듬어 배포하면 된다고 여기기 쉽다. 그러나 MVP의 가치는 작지만 완성된 제품이라는 데 있지 않다. 실제로 작동하는 대상을 보며 무엇이 맞고 무엇이 어긋나는지 확인할 수 있다는 데 있다.

첫 결과물이 나오기 전에는 사용자도 자신의 기준을 모두 알지 못할 수 있다. 결과물을 본 뒤에야 “이 방향은 아니다”, “이 기능은 필요 없다”, “이 경계는 넘으면 안 된다”고 말할 수 있다. LLM 역시 사용자가 되짚어 설명하기 전에는 말 뒤에 놓인 우선순위를 충분히 알 수 없다.

따라서 수정은 첫 결과물의 실패를 뜻하지 않는다. 서로 다르게 이해한 부분을 눈앞에 꺼내 놓고, 실제 판단을 시작했다는 증거일 수 있다.

첫 결과물을 완성으로 보면 수정은 뒷수습이 된다. 첫 결과물을 검증의 시작으로 보면 수정과 교정이 본 작업이 된다.

더 좋은 결과물보다 더 정확한 다음 질문

첫 결과물을 받은 뒤 곧바로 “더 좋게 만들어 줘”라고 요청하면 LLM은 일반적으로 좋아 보이는 방향으로 결과물을 확장한다. 설명을 풍부하게 하고, 기능을 더하며, 빠진 내용이 없도록 보완한다.

그러나 내용이 많아지는 것과 사용자의 목표에 가까워지는 것은 같은 일이 아니다. 사용자가 원한 것이 간결한 안내문이라면 자세한 설명은 오히려 핵심을 가릴 수 있다. 핵심 기능 하나를 시험하려는 MVP에 로그인·통계·알림 기능을 더하면 제품처럼 보이지만, 무엇을 검증하려 했는지는 흐려지고 수정할 범위만 커질 수 있다. 이처럼 결과물의 일반적인 완성도는 높아져도, 사용자가 해결하려던 문제에서는 멀어질 수 있다.

따라서 “더 좋게”라고 요청하기 전에 다음 네 가지를 먼저 확인하는 편이 낫다.

1. LLM은 내가 해결하려는 문제를 무엇이라고 이해했는가?
2. 내가 가장 중요하게 여긴 기준이 결과물에 제대로 반영됐는가?
3. 새로 추가된 내용 중 목표와 관계없거나 처음 정한 범위를 벗어난 것은 무엇인가?
4. 어떤 조건이 충족되면 지금 단계를 마쳤다고 판단할 수 있는가?

이 질문들은 LLM만 점검하기 위한 목록이 아니다. 사용자가 자신의 기준을 더 분명하게 발견하는 과정이기도 하다. 처음부터 원하는 것을 빠짐없이 설명하기는 어렵다. 실제 결과물을 본 뒤에야 무엇이 중요하고, 무엇이 불필요하며, 어디까지 해야 충분한지가 선명해질 수 있다.

먼저 LLM이 이해한 목표를 설명하고, 사용자는 맞게 이해한 부분과 어긋난 부분을 구분해 알려 준다. 그런 다음 남은 차이와 완료 조건을 함께 확인한다. 이 과정을 거치면 첫 결과물은 성급한 완성본이 아니라, 다음 판단을 함께 내리기 위한 구체적인 기준이 된다.

최근의 첫 결과물을 다시 열어 보기

최근 LLM과 만든 문서·계획·코드·이미지 가운데 하나를 고른다. 가장 완성도가 높은 것보다 “이 정도면 됐다”고 빨리 닫았던 결과물이 더 알맞다.

그 결과물에 다음 네 줄을 덧붙여 본다.

- 원래 해결하려던 문제:
- 결과물이 실제로 향한 목표:
- 빠졌거나 넘어서는 경계:
- 다시 확인할 질문:

네 줄이 쉽게 써진다면 초기 대화에서 목표와 기준이 비교적 잘 드러난 것이다. 쓰기 어렵다고 해서 작업이 실패한 것은 아니다. 첫 결과물의 매끄러운 외형이 가리고 있던 차이를 이제 발견한 것이다.

완성은 결과물이 매끄러워지는 순간 자동으로 찾아오지 않는다. 처음 제기한 문제가 충분히 해결되었는지 문제 제기자가 근거를 들어 판단할 수 있을 때, 비로소 한 단계를 마무리할 수 있다. 그 판단을 가능하게 하는 것은 완벽한 첫 프롬프트가 아니라 첫 결과물 이후에 이어지는 더 정확한 대화다.

첫 결과물이 곧 완성은 아니다. 그것은 서로의 이해와 실제 목표 사이의 차이를 처음 확인할 수 있게 해 주는 판단의 출발점이다.

독자 기록

- 나는 어떤 모습의 결과물을 보면 너무 빨리 완성됐다고 판단하는가?
- 결과물의 외형과 문제 해결을 혼동했던 최근 사례는 무엇인가?
- 다음 작업에서는 첫 결과물 뒤에 무엇을 반드시 확인할 것인가?

제2장 — 친절한 답이 목표를 잃게 할 때

ORBIT의 중심 질문을 처음 정리할 때, LLM은 문제를 이렇게 이해했다.

결과를 더 빠르게 만들 수 있게 된 인간은 무엇을 기준으로 만들 것인가?

문장만 보면 그럴듯했다. AI가 결과를 빠르게 만드는 시대에 인간의 판단 기준을 묻는 질문이었고, 철학 프로젝트의 출발점으로도 손색이 없어 보였다. 그러나 사용자가 중요하게 여긴 문제는 속도 자체가 아니었다.

사용자는 중심을 다시 짚었다.

“가장 큰 문제는 서로의 생각이 정확하게 전달되지 않는다는 점, 그리고 LLM의 지나친 친절이 방향을 흐리거나 목표를 넘어 실제로 원하는 결과에 닿지 못하게 할 수 있다는 점이겠지.”

편집 주: 대화 원문의 의미를 유지하며 문장을 다듬었다.

LLM의 첫 정리는 틀린 정보를 담은 답이 아니었다. 다만 사용자의 문제의식 가운데 눈에 잘 띄는 한 표현을 중심으로 삼았고, 더 근본적인 질문을 뒤로 밀어냈다. 문장은 좋아 보였지만 방향은 달랐다.

좋은 의도와 좋은 결과는 같은 말이 아니다

LLM은 대체로 도움을 주는 방향으로 답을 확장한다. 하나를 요청하면 관련 정보와 선택지를 더하고, 예상되는 위험을 경고하며, 사용자가 미처 생각하지 못했을 대안도 제안한다. 이런 친절 덕분에 놓칠 뻔한 문제를 발견하거나 더 나은 선택지를 만날 수도 있다.

따라서 문제는 LLM이 친절하다는 사실 자체가 아니다. 무엇을 위한 친절인지 확인되지 않은 채 확장이 계속되는 것이 문제다.

사용자는 간단한 비교표를 원했는데 배경 이론과 구매 가이드까지 붙을 수 있다. 수업에서 바로 쓸 활동지를 요청했는데 장기 교육과정과 평가 계획이 함께 제안될 수 있다. 핵심 기능을 시험할 MVP를 만들고 있는데 로그인, 통계, 알림과 관리 화면이 추가될 수도 있다. 각각만 보면 유용한 기능과 정보다. 하지만 지금 해결하려는 문제에는 필요하지 않을 수 있다.

그래서 결과물은 더 좋아 보이지만 사용자의 목표에서는 오히려 멀어질 수 있다. 여기서 “더 좋아 보인다”는 말은 내용이 많고 기능이 다양하며 설명이 매끄럽다는 뜻이다. 반면 목표에 가깝다는 것은 지금 풀어야 할 문제를 정해 둔 범위와 시간 안에서 해결한다는 뜻이다. 두 기준은 같지 않다.

예를 들어 MVP의 목적이 “사용자가 핵심 기능을 실제로 원하는지 확인하는 것”이라면, 로그인과 통계 화면을 더한 결과물은 제품처럼 보이게 한다. 하지만 개발 시간이 늘고 핵심 기능에 대한 검증은 늦어진다. 겉모습의 완성도는 높아졌어도 MVP를 만든 목적에서는 멀어진 것이다.

도움이 되는 내용을 더하는 과정에서 처음 목표가 흐려지면 친절은 방향 이탈로 바뀐다. LLM의 기준에서는 결과가 더 풍부해졌지만, 사용자의 기준에서는 범위와 판단 부담이 함께 커진 셈이다.

정보가 부족해서가 아니라 너무 많아서 생기는 문제

우리는 부족한 답을 경계하는 데 익숙하다. 설명이 빠졌는지, 위험을 놓쳤는지, 더 나은 대안이 있는지 확인한다. 반면 답이 너무 많은 가능성을 담을 때 생기는 문제는 알아차리기 어렵다. 내용이 많을수록 더 성실한 답처럼 보이기 때문이다.

그러나 선택지가 늘어나면 판단의 부담도 커진다. 원래 비교하려던 기준보다 새로운 가능성을 검토하는 데 더 많은 시간을 쓰게 되고, 꼭 지켜야 할 조건이 여러 장점 사이에 묻힐 수 있다. 무엇을 할 수 있는지는 잘 보이지만 무엇을 해야 하는지는 오히려 흐려진다.

좋은 답은 가능한 내용을 모두 담은 답이 아니다. 지금의 목표에 필요한 것은 남기고, 필요하지 않은 것은 덜어낼 수 있는 답이다.

펼치는 일과 덜어내는 일

LLM이 가능성을 넓게 펼치는 능력은 협업에서 중요한 역할을 한다. 사용자는 자신이 알고 있는 범위 안에서만 질문할 수 있기 때문에, 예상하지 못한 대안과 위험을 만나는 일은 가치가 있다.

하지만 펼쳐진 가능성이 모두 목표가 되는 것은 아니다. 사용자는 자신의 상황에 맞지 않는 제안을 덜어내고, 감당할 수 없는 비용과 넘지 말아야 할 범위를 말해야 한다. LLM은 그 이유를 바탕으로 선택지를 다시 좁히고, 남은 방향 안에서 놓친 위험이 없는지 살필 수 있다.

이 과정을 “AI는 펼치고 인간은 덜어낸다”는 고정된 역할로 이해할 필요는 없다. 사용자도 새로운 가능성을 제안할 수 있고, LLM도 이미 정한 기준에 따라 불필요한 내용을 먼저 줄일 수 있다. 중요한 것은 무엇을 더했고 무엇을 뺐는지, 그 이유가 무엇인지 서로 알 수 있어야 한다는 점이다.

친절을 거절하지 않고 방향을 지키는 말

과잉 친절을 막는 방법은 “시키는 것만 해”라고 가능성을 닫는 것이 아니다. 제안과 실행을 구분하면 된다.

- 지금 요청한 범위 안에서 먼저 결과를 만든다.
- 범위를 넓힐 만한 아이디어는 본문에 섞지 않고 별도로 제안한다.
- 추가 제안이 목표·시간·비용에 어떤 영향을 주는지 설명한다.
- 사용자가 선택하기 전에는 새 기능이나 내용을 기본 범위로 간주하지 않는다.

이렇게 하면 LLM의 제안을 잃지 않으면서도 원래 작업의 방향을 지킬 수 있다. 친절은 많을수록 좋은 것이 아니라, 상대가 판단할 수 있는 형태로 건네질 때 도움이 된다.

최근의 친절한 답을 다시 살펴보기

최근 LLM이 요청보다 많은 내용을 제안했던 작업 하나를 고른다. 결과물에 다음 표시를 해 본다.

- 내가 처음 요청한 것:
- LLM이 추가로 제안한 것:
- 실제로 도움이 된 확장:
- 목표를 흐리거나 범위를 키운 확장:
- 다음에는 제안으로만 남겨 둘 것:

이 구분은 추가된 내용을 무조건 삭제하기 위한 것이 아니다. 무엇이 실제 목표에 기여했고 무엇이 단지 좋아 보였는지를 판단하기 위한 것이다.

친절은 방향을 대신할 수 없다. 좋은 협업은 가능성을 많이 내놓는 데서 끝나지 않고, 어떤 가능성을 왜 남기고 버렸는지 함께 확인하는 데서 시작된다.

독자 기록

- 유용해 보여서 받아들였지만 실제 목표에는 필요 없었던 제안이 있었는가?
- LLM이 제안과 실행을 구분하도록 요청한다면 어떤 문장을 사용할 것인가?
- 다음 작업에서 반드시 지키고 싶은 범위 한 가지는 무엇인가?

제3장 — 같은 말을 해도 같은 뜻은 아니다

사용자가 “간단하게 만들어 줘”라고 말한다. LLM도 요청을 이해했다고 답한다. 둘은 같은 문장을 보고 있지만 “간단하게”가 뜻하는 모습은 다를 수 있다.

사용자는 분량을 줄여 바로 사용할 수 있기를 바랐을 수 있다. LLM은 설명을 쉽게 풀어 쓰라는 뜻으로 이해해 문장은 쉬워졌지만 분량은 더 늘릴 수 있다. 요청은 전달됐고 결과도 자연스럽지만, 의미는 어긋난다.

이 문제는 특별히 모호한 단어에서만 생기지 않는다. “종계”, “학생 수준으로”, “핵심만”, “완성해 줘”, “실제로 쓸 수 있게”처럼 일상적으로 사용하는 표현에도 여러 기준이 겹쳐 있다. 대화에 같은 단어가 등장했다는 사실만으로 같은 뜻에 도달했다고 볼 수는 없다.

문장을 이해하는 것과 의도를 이해하는 것

LLM은 사용자가 쓴 문장에서 목표를 추론한다. 일반적인 사례와 표현의 관계를 바탕으로 가장 가능성 높은 뜻을 선택하고 결과를 만든다. 이 능력 덕분에 사용자가 모든 조건을 설명하지 않아도 작업을 빠르게 시작할 수 있다.

그러나 추론은 확인이 아니다. 사용자가 말하지 않은 조건까지 자동으로 알 수는 없다.

“초등학생 수준”이라는 요청을 생각해 보자. 쉬운 낱말을 사용하라는 뜻일 수도 있고, 개념의 난도를 낮추라는 뜻일 수도 있다. 한 문장을 짧게 쓰라는 뜻일 수도 있으며, 학생이 직접 해 볼 활동을 중심에 놓으라는 뜻일 수도 있다. LLM이 그중 하나를 선택해 매끄러운 결과를 만들면, 다른 기준은 빠질 수 있다.

사용자 역시 처음부터 자신의 뜻을 완전하게 알고 있는 것은 아니다. 결과물을 본 뒤에야 “낱말은 쉬운데 설명이 너무 길다”, “내용은 맞지만 우리 학생들이 직접 하기

어렵다”고 말할 수 있다. 의미의 어긋남은 LLM이 잘못 추론해서만 생기는 것이 아니다. 결과를 통해 사용자 자신의 기준이 뒤늦게 드러나기 때문에도 생긴다.

이해했다는 말보다 이해한 내용을 보여 주기

“이해했습니다”라는 답은 대화를 빠르게 이어 가게 하지만, 무엇을 어떻게 이해했는지는 보여 주지 않는다. 서로의 뜻을 확인하려면 동의의 표현보다 이해한 내용을 짧게 다시 말하는 편이 낫다.

예를 들어 LLM은 작업을 시작하기 전에 이렇게 되물을 수 있다.

제가 이해한 목표는 초등학생이 수업 시간 안에 직접 사용할 수 있는 짧은 활동지를 만드는 것입니다. 설명을 쉽게 쓰되 분량을 늘리지 않고, 교사용 이론과 확장 활동은 이번 범위에서 제외하겠습니다.

사용자는 이 문장에서 맞는 부분과 어긋난 부분을 바로 찾을 수 있다. “활동지가 아니라 발표 자료다”, “확장 활동 하나는 필요하다”, “학생용이지만 교사가 읽어 줄 것이다”처럼 수정하면 숨겨져 있던 기준이 드러난다.

되말하기는 사용자의 문장을 길게 반복하는 일이 아니다. 목표, 가장 중요한 기준, 하지 않을 것, 완료 상태를 상대가 확인할 수 있는 형태로 번역하는 일이다.

의미 확인은 한 번의 절차가 아니다

처음에 목표를 확인했다고 해서 작업이 끝날 때까지 같은 이해가 유지되는 것은 아니다. 결과물이 구체화되면 새로운 선택과 경계가 생긴다. 사용자는 처음에는 중요하지 않다고 생각했던 조건을 발견할 수 있고, LLM은 새 정보 때문에 이전 제안을 바꿀 수 있다.

따라서 의미 확인은 다음과 같은 짧은 순환에 가깝다.

1. LLM이 이해한 목표와 기준을 다시 말한다.
2. 사용자가 맞는 부분과 어긋난 부분을 구분한다.
3. 첫 결과물을 보며 새로 발견한 기준을 더한다.
4. 수정된 이해와 아직 남은 차이를 다시 확인한다.

이 과정은 완벽한 합의를 만들기 위한 것이 아니다. 서로 다른 생각을 하나로 합치는 것도 아니다. 같은 결론에 도달하지 않더라도, 상대가 무엇을 중요하게 여기고 어디에서 생각이 다른지 설명할 수 있다면 의미 확인은 이루어진다.

대화 깔때기와 통역

처음 대화에는 여러 해석과 가능성이 섞여 있다. 질문을 주고받으며 필요하지 않은 해석을 덜어내고, 목표에 맞는 기준만 남기면 선택지는 점차 좁아진다. 이 모습은 넓은 입구에서 좁은 출구로 향하는 깔때기와 닮았다.

동시에 이 과정은 통역이기도 하다. 한국어를 다른 언어로 옮기는 통역이 아니라, 인간의 경험과 우선순위를 LLM이 작업할 수 있는 기준으로 바꾸고, LLM의 추론과 제안을 사용자가 판단할 수 있는 말로 되돌리는 통역이다.

통역은 어느 한쪽만의 책임이 아니다. 사용자는 LLM이 마음을 읽으리라 기대할 수 없고, LLM은 자연스러운 결과를 만들었다는 이유로 사용자의 뜻을 모두 이해했다고 단정할 수 없다. 서로가 이해한 내용을 보여 주고, 상대가 다르다고 말하면 다시 고쳐야 한다.

다음 작업 전에 네 줄로 확인하기

중요한 작업을 시작하기 전, 긴 프롬프트를 새로 쓰는 대신 다음 네 줄을 함께 확인해 볼 수 있다.

- 지금 해결하려는 문제:
- 가장 중요하게 지킬 기준:
- 이번에는 하지 않을 것:
- 이 단계가 끝났다고 판단할 조건:

네 줄은 모든 요구를 고정하는 계약서가 아니다. 작업 도중 기준이 바뀌면 이유와 함께 고칠 수 있다. 중요한 것은 처음의 문장을 끝까지 지키는 것이 아니라, 서로의 이해가 어떻게 달라졌는지 알 수 있게 하는 것이다.

같은 말을 사용하는 것과 같은 뜻에 도달하는 것은 다르다. 좋은 대화는 오해가 없는 대화가 아니라, 어긋남을 확인하고 다시 맞출 수 있는 대화다.

독자 기록

- 내가 자주 쓰지만 사람마다 다르게 이해할 수 있는 표현은 무엇인가?
- 최근 LLM과의 작업에서 같은 단어를 다르게 이해했던 순간이 있었는가?
- 다음 중요한 요청에서 LLM에게 무엇을 되말해 달라고 할 것인가?